

Übungsblatt 2

Git, CSS und Evolution des Webs

5430UE Web and Data Engineering

22. April 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

Lernziele

- Grundfunktion der Versionsverwaltung Git anwenden
- CSS Selektoren formulieren
- Evolution des Webs (1.0, 2.0, 3.0) verstehen

Git

Übung 2.1: FIM Account & FIM Gitlab

Für den Zugang auf das FIM Gitlab benötigen Sie einen FIM Account (siehe Blatt 01).

1. Stellen Sie sicher, dass Sie Zugang zum FIMGit haben. Melden Sie sich dafür mit Ihrer FIM-Kennung unter [FIMGit](#) an.
2. Machen Sie sich mit der Oberfläche vertraut. Sie werden später FIMGit nutzen, um eigene Repositories anzulegen.

Übung 2.2: Einführung Git

Für die kommende Projektarbeit sollen Sie sich mit dem Versionsverwaltungs-Tool **Git** vertraut machen. Um ein Verständnis für die wichtigsten Befehle zu erhalten, arbeiten Sie das verlinkte [Tutorial](#) durch.

Wenn Sie sich keinen Bitbucket-Account anlegen wollen, können Sie auch direkt mit Ihrem FIMGit arbeiten. Erstellen Sie dazu das notwendige Repository in Ihrem FIMGit (unter *Projects*).

Voraussetzung: Falls Sie auf Ihrem System noch kein **Git** installiert haben, finden Sie unter <https://git-scm.com/> den entsprechenden Download sowie Installationsanleitungen für Ihr System.

Weitere interaktive Übungsmöglichkeit: Bei weiterem Übungsbedarf können Sie zusätzlich das verlinkte [interaktive Tutorial](#) absolvieren.

Übung 2.3: Verwendung von FIMGit

1. Erstellen Sie in FIMGit ein neues Repository `wde-exercise01`.
2. Klonen Sie das Repository lokal.
3. Erstellen Sie die Datei `hello.txt` mit dem Inhalt *Hallo WDE!*, committen Sie die Datei und pushen anschließend den Commit. Notieren Sie alle für diese Teilaufgabe notwendigen Befehle!

Übung 2.3: Verwendung von FIMGit — Lösung

Teilaufgabe 3:

```
echo "Hallo WDE!" > hello.txt  
git add hello.txt  
git commit -m "Add hello.txt"  
git push origin main
```

Warum diese Reihenfolge?

- `git add` überführt die Datei in den Staging-Bereich
- `git commit` erzeugt den Snapshot
- `git push` überträgt den Commit an den Remote-Server

Übung 2.4: Git Grundlagen - Branching

Angenommen Sie arbeiten an einem Feature user-login in einem Projekt.

1. Geben Sie die Git-Befehle an, die nötig sind, um einen neuen Branch feature/user-login zu erstellen und direkt zu wechseln?
2. Erklären Sie kurz den Unterschied zwischen `git merge` und `git rebase`?
3. Beim GitFlow-Prinzip wird empfohlen, Arbeiten in Feature-Banches auszulagern statt direkt auf dem main-Branch zu arbeiten. Erklären Sie kurz, warum das sinnvoll ist.

Übung 2.4: Branching — Lösung (1/2)

1. Erstellung des Branches:

```
git checkout -b feature/user-login  
# oder modern:  
git switch -c feature/user-login
```

2.
 - `git merge`: Kombiniert Änderungen aus dem Quell-Branch in den Ziel-Branch. Git erstellt dabei einen speziellen Merge-Commit, der zwei Eltern-Historien miteinander verknüpft.
 - `git rebase`: Nimmt die Commits des Feature-Branche und setzt sie "oben auf" auf die aktuelle Spitze des Ziel-Branche. Es ist so, als hätten die Arbeit gerade erst auf Basis der neuesten Änderungen begonnen.

Übung 2.4: Branching — Lösung (2/2)

3. Feature-Branches isolieren Arbeiten voneinander:

- main-Branch bleibt jederzeit deploybar
- Parallele Features stören sich nicht
- Code-Reviews über Merge Requests sind vor dem Merge möglich

Evolution des Webs

Übung 2.5: Evolution des Webs — 1.0, 2.0, 3.0

Das Web hat sich in Generationen entwickelt. Durchforsten Sie das Internet nach Informationen zu den Bereichen **Web 1.0**, **Web 2.0** sowie **Web 3.0**.

Füllen Sie die Tabelle aus und erklären Sie welches Web-Paradigma heute in Unternehmensanwendungen dominiert.

Merkmal	Web 1.0	Web 2.0	Web 3.0
Inhaltsproduktion			
Nutzerinteraktion			
Typische Technologien			
Beispiel-Anwendung			

Übung 2.5: Evolution des Webs — Lösung (1/2)

Merkmal	Web 1.0	Web 2.0	Web 3.0
Inhaltsproduktion	Nur Betreiber	Nutzer generieren Inhalte (UGC)	Dezentralisiert, maschinenlesbar
Nutzerinteraktion	Passiv (Lesen)	Aktiv (Teilen, Kommentieren)	Semantisch, personalisiert, KI-gestützt
Typische Technologien	Statisches HTML, FTP	AJAX, RSS, Web-APIs, Blogs	Linked Data, RDF, KI/ML-Integration
Beispiel-Anwendung	Unternehmens-Broschüre-Website, persönliche Homepages	Facebook, Wikipedia, YouTube, Instagram	Semantische Suchmaschinen, KI-Assistenten

Übung 2.5: Evolution des Webs — Lösung (2/2)

Dominierendes Web-Paradigma:

Heute dominiert das Web-2.0-Paradigma in Unternehmensanwendungen: interaktive Single-Page Applications, REST-APIs, nutzererzeugte Daten. Web-3.0-Elemente (semantische Metadaten, KI-Empfehlungen) werden zunehmend integriert.

CSS

Übung 2.6: Wiederholung CSS

Aus der Veranstaltung *Einführung in Internet Computing* kennen Sie bereits die Grundlagen von CSS.

Bearbeiten Sie die 32 Level aus dem CSS Dinner <https://flukeout.github.io/>, um Ihre Kenntnisse aufzufrischen und die Grundlagen für CSS Selektoren zu schärfen.

Übung 2.6: Wiederholung CSS — Lösung (1/4)

1. Auswahl nach Typ des Elements (Hier: plate): `plate`
2. Auswahl nach Typ des Elements (Hier: bento): `bento`
3. Auswahl eines Elements mit ID (Hier: Element mit ID fancy): `#fancy`
4. Nachfolger Elemente auswählen - Ein Element innerhalb eines anderen Elements auswählen (Hier: Wählt alle apple-Elemente, die Nachfahren (descendants) eines plate sind.): `plate apple`
5. Kombination von Nachfolger und ID (Alle pickle innerhalb des Elements mit ID fancy): `#fancy pickle`
6. Auswahl aller Elemente mit Klasse (Hier: Klasse small): `.small`
7. Kombination von Klasse und Typ - alle Elemente des Typs, die die Klasse besitzen (Hier: Alle orange mit Klasse small): `orange.small`
8. Kombination bisheriger Inhalte: Wähle kleine Orangen in Bento-Boxen: `bento orange.small`

Übung 2.6: Wiederholung CSS — Lösung (2/4)

- 9. Liefert alle Elemente der aufgeführten Typen (Hier: plate und bento): `plate, bento`
- 10. Wähle alle Elemente: `*`
- 11. Wähle alle Elemente auf Tellern: `plate *`
- 12. Selektor für benachbarte Geschwisterelemente - Wähle ein Element aus, das direkt auf ein anderes Element folgt (Hier: Wählt ein apple, das direkt auf ein plate folgt (adjacent sibling).): `plate + apple`
- 13. Allgemeiner Geschwister-Selektor Wählt Elemente aus, die auf ein anderes Element folgen (Hier: Wählt alle pickle, die Geschwister von bento sind und nach diesem kommen.): `bento ~ pickle`
- 14. Wähle direkte Kinder (Hier: Äpfel auf Tellern): `plate > apple`
- 15. Erstes Kindelement innerhalb eines anderen Elements (Hier: Wähle Orangen, die erstes Kind sind): `orange:first-child`
- 16. Elemente, die einziges Kind eines Elements sind (Hier: Wählt plate, die selbst das einzige Kind ihres Eltern-Elements sind.): `plate:only-child`

Übung 2.6: Wiederholung CSS — Lösung (3/4)

- 17. Elemente, die das letzte Element innerhalb eines anderen Elements sind (Hier: Wähle kleine Elemente als letztes Kind): `.small:last-child`
- 18. Wähle jedes dritte Element innerhalb eines anderen Elements: `:nth-child(3)`
- 19. Wähle ein Element anhand seiner Position innerhalb eines anderen Elements aus, wobei von hinten gezählt wird (Hier: Wähle Element vom Typ Bento, die drittletztes Kind sind): `bento:nth-last-child(3)`
- 20. Wählt erstes Element eines bestimmten Typs (Hier: Wähle ersten Apfel): `apple:first-of-type`
- 21. Wähle gerade Teller: `plate:nth-of-type(even)`
- 22. Wählt alle plate, deren Position der Formel $2n+3$ entspricht (3., 5., 7., ...).d.h. jedes zweite plate, startend beim dritten plate: `plate:nth-of-type(2n+3)`
- 23. Wähle Elemente aus, die innerhalb ihres übergeordneten Elements die einzigen ihres Typs sind (Hier: Wähle apple, wenn apple einziges Element seines Typs innerhalb von plate ist): `plate apple:only-of-type`
- 24. Wähle letztes Element vom Typ apple und vom Typ orange: `apple:last-of-type, orange:last-of-type`

Übung 2.6: Wiederholung CSS — Lösung (4/4)

- 25. Wählt Elemente ohne Kindelemente (Hier: bento Elemente ohne Kindelemente): `bento:empty`
- 26. Wähle alle Elemente aus, die nicht dem Negationsselektor entsprechen: (Hier: Wähle apple ohne Klasse `small`): `apple:not(.small)`
- 27. Wähle alle Elemente mit Attribut „for“: `*[for]`
- 28. Wähle alle plate Elemente mit „for“-Attribut: `plate[for]`
- 29. Wähle bento Elemente, deren „for“-Attribut den Wert "Vitaly" besitzt: `bento[for="Vitaly"]`
- 30. Wähle Elemente, deren Attributwert des „for“-Attributs mit „Sa“ beginnt: `*[for^="Sa"]`
- 31. Wähle alle Elemente, deren Attributwert des „for“-Attributs mit „ato“ endet: `*[for$="ato"]`
- 32. Wähle Elemente, deren Attributwert des „for“-Attributs „obb“ enthält: `*[for*="obb"]`

Übung 2.7: CSS - Anwendung (1/2)

Betrachten Sie folgenden HTML-Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>TODO Liste</title>
</head>
<body>
  <h1>Aufgaben:</h1>
  <p>Folgende Aufgaben muessen <strong>unbedingt</strong> erledigt werden.</p>
  <ul>
    <li class="styles" for="basic" id="entry01">Aufraeumen<input type="checkbox" checked></li>
    <li>Einkaufen</li>
    <li for="basicStudies">Lernen
      <ol>
        <li id="entry02">HTML<input type="checkbox" checked></li>
        <li>CSS<input type="checkbox"></li>
      </ol>
    </li>
    <li for="peace">Entspannen<input type="checkbox" checked></li>
  </ul>
</body>
</html>
```

Übung 2.7: CSS - Anwendung (2/2)

Erweitern Sie den <head> um ein Style-Element, mit denen Sie folgende Darstellungsänderungen durchführen:

1. Erhöhen Sie die Schriftgröße für alle Listeneinträge ohne Klasse "styles" auf 20 Pixel.
2. Blenden Sie das zweite Element der TODO-Liste und der Unterliste "Lernen" aus.
3. Färben Sie "unbedingt" in Rot.
4. Drehen Sie den "Aufgaben:" Header um 10 Grad an der Z Achse. Verschieben Sie den Header um 40 Pixel nach rechts und 150 Pixel nach unten.
5. Streichen Sie den Text mit Attributen "basic" und "basicStudies" mit Hilfe eines einzelnen Selektors durch.

Übung 2.7: CSS - Anwendung — Lösung

```
li:not(.styles) {  
  font-size: 20px;  
}  
  
li:nth-child(2) {  
  display: none;  
}  
  
strong {  
  color: red;  
}  
  
h1 {  
  transform: rotateZ(10deg) translate(40px, 150px);  
}  
  
li[for*="basic"] {  
  text-decoration: line-through;  
}
```

Materialien

Materialien zum Übungsblatt

- [todo list template.html](#)